

JEDDY

Analyse de ton fichier config/database.php

✓ Tes points forts

Utilisation du typage PHP : Tu as précisé le type de retour : PDO. C'est une très bonne habitude qui rend ton code plus robuste et aide les outils de développement à te suggérer les bonnes méthodes par la suite.

Configuration du Fetch Mode : En définissant PDO::FETCH_ASSOC par défaut dès la connexion, tu simplifies ton code futur. Tes modèles n'auront plus besoin de le préciser à chaque requête.

Sécurité et Charset : Tu as bien inclus charset=utf8 dans ton DSN, ce qui t'évitera bien des soucis avec les caractères spéciaux.

Documentation : Ton code est commenté de manière concise, ce qui est toujours apprécié pour la maintenance.

Tes points à améliorer

Gestion du die() : Comme pour tes camarades, évite d'utiliser die() avec le message d'erreur brut en production. Cela peut exposer des informations sensibles sur ton infrastructure.

Identifiants en dur : Pour passer au niveau supérieur, tu pourrais envisager de sortir ces variables (\$host, \$user, etc.) dans un fichier de configuration séparé ou des variables d'environnement.

Instance multiple : À chaque fois que tu appelles getConnection(), tu crées une nouvelle connexion. Dans un gros projet, on préfère souvent utiliser une variable statique pour réutiliser la même connexion tout au long de la page.

Analyse de ton fichier `models/Contact.php`

✓ Tes points forts

Utilisation du typage strict : Tu as défini le type de ta propriété (PDO \$pdo) et les types de retour de tes méthodes (array, bool). C'est une excellente pratique qui limite les bugs en amont.

Chemin d'inclusion sécurisé : L'utilisation de `__DIR__` pour le `require_once` est parfaite. Cela garantit que le fichier sera trouvé quel que soit l'endroit d'où le script est exécuté.

Traitement des données au bon endroit : Tu as choisi d'appliquer `trim()` et `htmlspecialchars()` directement dans le modèle lors de l'insertion. C'est une stratégie intéressante pour garantir que les données en base sont toujours "propres".

Requêtes préparées : Tu maîtrises bien l'usage des marqueurs nommés (`:name`, `:email`) pour la sécurité.

Tes points à améliorer

Double emploi du `htmlspecialchars()` : Attention, en appliquant `htmlspecialchars()` à l'insertion (dans le modèle), tu stockes des entités HTML dans ta base de données (ex: `é` au lieu de `é`). La convention habituelle est plutôt de stocker les données brutes et d'appliquer `htmlspecialchars()` uniquement à l'affichage (dans la vue).

Validation de l'email : Comme tes camarades, tu ne vérifies pas encore si la chaîne `$email` est un format valide avant de tenter l'insertion.

Gestion des erreurs : Ta méthode `add()` renvoie un booléen. C'est correct, mais si l'email doit être unique en base et qu'un doublon survient, le contrôleur ne saura pas *pourquoi* l'insertion a échoué.

Analyse de ton ContactController.php

✓ Tes points forts

Validation complète : C'est un excellent point. Contrairement à d'autres, tu vérifies non seulement si le champ est vide, mais tu valides aussi le format de l'email avec `filter_var`.

Gestion des erreurs : Ton approche consistant à collecter les erreurs dans un tableau (`$errors`) et à réafficher la vue avec ces messages est très professionnelle. Cela permet à l'utilisateur de comprendre exactement ce qui a échoué.

Sécurité des types : Le cast explicite `(int) $_GET['id']` dans la méthode `delete` est un réflexe de sécurité indispensable pour éviter de passer des données corrompues au modèle.

Utilisation du typage PHP 8 : L'usage du type de retour `void` et du typage de la propriété `private Contact $model` rend ton code moderne et rigoureux.

Tes points à améliorer

Logique de réaffichage : En cas d'erreur dans la méthode `add`, tu récupères à nouveau tous les contacts (`$this->model->getAll()`) pour afficher la vue. C'est correct, mais cela crée une légère duplication de logique avec la méthode `list`. Tu pourrais envisager une méthode privée de rendu pour centraliser cela.

Couplage : Tu instances ton modèle directement dans le constructeur. Comme nous l'avons vu pour tes camarades, c'est tout à fait acceptable à ce niveau, mais garde en tête qu'un système plus complexe bénéficierait d'une injection de dépendance.

Analyse de ton fichier views/contacts.php

✓ Tes points forts

Design "High-End" : L'utilisation des polices Google Fonts (*Syne* et *DM Sans*), des variables CSS pour gérer ton thème et le système de grille (*CSS Grid*) montre une excellente maîtrise du front-end moderne.

Sécurité irréprochable : Tu as appliqué `htmlspecialchars()` partout, y compris pour réafficher les valeurs du formulaire en cas d'erreur (`value="<?= ... ?>"`). C'est le niveau de détail qui sépare un étudiant d'un développeur junior prêt pour l'emploi.

Gestion des états : Tu as prévu un "Empty State" (affichage si la liste est vide), ce qui évite d'afficher un tableau vide et améliore grandement l'expérience utilisateur.

Récupération des données : Le fait de conserver les valeurs saisies par l'utilisateur dans les champs input après une erreur de validation est un énorme plus pour l'ergonomie.

Tes points à améliorer

La balise de soumission : Dans ton formulaire, tu as utilisé une balise `<a>` avec une classe `btn-add` au lieu d'un `<button type="submit">`. Pour que ton formulaire soit envoyé correctement sans JavaScript, il est préférable d'utiliser un bouton de type `submit`.

Double échappement : Puisque tu utilises déjà `htmlspecialchars()` dans ton modèle lors de l'insertion, les données arrivent déjà "nettoyées" dans ta base. En le refaisant dans la vue, tu pourrais avoir des problèmes d'affichage si des caractères spéciaux complexes sont utilisés. La bonne pratique est de stocker les données brutes et de n'échapper qu'à l'affichage.

Fiche d'Évaluation : Projet Répertoire MVC

Étudiant : Jeddy

Projet : Gestionnaire de Contacts (Style Neo-Minimalist)

Note Globale : 19,5 / 20

Détail de l'Évaluation

Critère	Note	Observations
Architecture MVC	5 / 5	Découpage parfait. Utilisation rigoureuse des dossiers et des responsabilités.
Sécurité & Rigueur	5 / 5	Validation d'email, typage strict, protection XSS et requêtes préparées. C'est exemplaire.
Qualité du Code (PHP)	5 / 5	Utilisation moderne de PHP (types de retour, propriétés typées). Code très lisible.
Interface & UX (UI)	4,5 / 5	Design magnifique et très "pro". Un petit point de moins pour le bouton du formulaire qui mériterait d'être un vrai submit.

Pour tes prochains projets, adopte la règle : "**Données brutes en base, données protégées à l'affichage**". Cela te facilitera la vie si tu dois un jour exporter tes données vers un autre format (PDF, Excel).